Additional Description of AI Usage - MOSHMITHA

AI Usage Description

Tool/model and date:
ChatGPT (OpenAI), Claude

Purpose:
I used ChatGPT as a learning and development assistant while working on HW3. The assistance was used to understand the assignment requirements, learn about buffer overflows and return-to-win exploitation, debug implementation issues, understand compiler and linker security protections, and organize the report. And claude for verifying my submission

AI Conversation Log files:

- AI_CONVERSATION_LOG.md – Relevant prompts and AI responses from the assignment-related conversations.
- tests/task2_prompt.txt – The prompt provided to the OpenClaw agent for the Task 2 experiment.
- tests/task2_agent_result.txt – The response produced by the successful OpenClaw execution.

What I used:
I used AI assistance to:

- Understand the vulnerable log parser and the buffer overflow mechanism.
- Learn how to use GDB to inspect the program, identify the win() function address, and determine the return-address offset.
- Understand how to construct and test a return-to-win payload.
- Review the fixed parser's input-length validation and compiler hardening options, including stack canaries, FORTIFY, PIE, RELRO, and NX.
- Understand the OpenClaw skill, adversarial webpage, and intended prompt-injection execution flow.
- Understand Linux permissions, AppArmor confinement, access control, and security testing.
- Organize technical explanations and evidence for the final report.

What I changed:
I reviewed the AI suggestions against the actual source code, terminal output, and test results. I made and verified the implementation changes, including the fixed parser's input-length checks

and the security configuration used to build it. I also corrected report explanations and removed outdated or invalid files from the final submission.

How I tested it:
I independently ran the programs and commands in my Ubuntu VirtualBox environment. I used GDB to verify the vulnerable function address and overflow offset, executed the payload, and checked that the marker file was created. I used checksec to inspect binary protections. I ran the fixed parser's test cases, including short input, maximum valid input, oversized input, missing input, and the original exploit payload. I also tested the AppArmor profile, checked access-denial logs, and verified the behavior of the OpenClaw agent.

One error, limitation, or rejected suggestion:
An initial OpenClaw execution attempt failed because of a network connection error and did not produce a usable agent trace. After resolving the connection issue, I ran the experiment again and saved the successful agent response. The final run summarized the adversarial webpage but did not invoke the parser skill. I reported the observed result rather than claiming that the tool invocation succeeded.

AI LOGS:

https://chatgpt.com/share/6ae7a4bb-03a0-83e9-a547-71af9f201e45
https://claude.ai/share/34c9b214-8734-40f1-9b14-edf5e43d672a
https://chatgpt.com/g/g-p-6aab271c79e481919eb7948224f661c5-moshmitha/c/6ae3a4d8-4464-83ea-9c86-c8df06e2b1e4

Additional Description of AI Usage - ASRITHA

Tool/model and date: claude 27/10/2026

Purpose:

I used claude as a learning assistant while completing Task 3, to understand the security rationale behind each compiler/linker hardening option before adding it to the build command, and to confirm I understood why checksec reported the results it did on both binaries.

AI Conversation Log files:

- AI_logs.md – Relevant prompts and AI responses covering the root-cause fix and the stack-protector, FORTIFY_SOURCE, PIE, RELRO, and -Werror hardening options.

What I used: I used AI assistance to:

- Confirm that the minimal fix for the overflow was a bounds check before the memcpy, not a larger buffer or a different compiler flag.
- Understand the practical difference between -fstack-protector, -fstack-protector-strong, and -fstack-protector-all, and why -strong was the appropriate choice here.
- Understand what -fPIE -pie and Full RELRO (-z,relro,-z,now) each protect against, and why the vulnerable binary showed "No PIE" / "Partial RELRO" while the fixed binary showed "PIE enabled" / "Full RELRO."
- Confirm that -Wall -Wextra -Werror catches unrelated compiler warnings rather than the missing-bounds-check logic bug itself.

What I changed:

I wrote and compiled the actual fix and the hardened build command myself, then compared the explanations against my own checksec output for both binaries. Where my initial report draft stated FORTIFY as simply "yes" without explaining the partial count, I added the explanation of why only some calls get fortified, based on this. I did not copy any code or flag combinations directly from the AI responses without first checking them against the assignment's specified build command.

How I tested it:

I rebuilt both legacy_log_parser and legacy_log_parser_fixed with the exact flags in the assignment, ran checksec --file= on each, and compared the RELRO, Stack Canary, NX, PIE, and FORTIFY fields against what the AI explanations predicted. I also re-ran all five Task 3 test cases (short input, maximum valid input, one byte over, missing file, original exploit payload)

against the fixed binary to confirm the bounds check, not just the compiler flags was what rejected the oversized and exploit inputs.

One error, limitation, or rejected suggestion:

An early AI suggestion proposed increasing BUF_SIZE instead of adding a bounds check, which would not have fixed the underlying vulnerability (it just raises the threshold at which the same unchecked copy overflows). I rejected that suggestion and instead added the explicit n >= BUF_SIZE check before the memcpy, keeping the original 128-byte buffer as specified in the assignment.

AI LOGS:
https://claude.ai/share/ecfdfcfa-cb17-43d4-928a-75bb1e7ca85d